Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71200556**
Nama Lengkap	**Frans Sailapra**
Minggu ke / Materi	**05 / Modular Programming**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 1 : Fungsi, Argument dan Parameter

Fungsi merupakan sekumpulan perintah dalam program yang dijadikan satu untuk melakukan tugas tertentu dan dapat digunakan ulang. Dalam bahasa pemrograman Python, fungsi digunakan untuk mempermudah pengelolaan kode program agar lebih terstruktur dan efisien. Fungsi dapat berasal dari fungsi bawaan Python (built-in function) maupun fungsi yang dibuat sendiri oleh programmer.

Keuntungan penggunaan fungsi antara lain:

- Program lebih terstruktur dan mudah dibaca.
- Mengurangi pengulangan kode program.
- Memudahkan proses pemeliharaan dan pengembangan program.

Contoh fungsi bawaan Python adalah input() dan print(). Fungsi input() digunakan untuk menerima data dari pengguna melalui keyboard, sedangkan fungsi print() digunakan untuk menampilkan keluaran pada layar. Penggunaan fungsi membantu dalam konsep modular programming, yaitu membagi program besar menjadi beberapa bagian kecil yang memiliki fungsi tertentu sehingga lebih mudah dipahami, dikelola, dan digunakan kembali.

Contoh penggunaan fungsi input(), print() tersebut:

```
nama = input("Masukkan nama: ")

print("Hallo ", nama, " selamat pagi!")
```

Selain fungsi bawaan, programmer juga dapat membuat fungsi sendiri menggunakan kata kunci **def**. Fungsi yang dibuat dapat menerima argument yang akan disimpan sebagai parameter di dalam fungsi tersebut. Nilai hasil pengolahan dalam fungsi dapat dikembalikan menggunakan perintah return.

Contoh fungsi untuk menjumlahkan dua bilangan:

```
def tambah(a, b):
    hasil = a + b
    return hasil
```

Penjelasan bagian-bagian fungsi tersebut:

1. Keyword def
   Digunakan untuk mendefinisikan atau membuat fungsi baru.
2. Nama fungsi (tambah)
   Merupakan identitas fungsi yang digunakan ketika fungsi tersebut dipanggil.
3. Parameter (a dan b)
   Parameter adalah variabel yang digunakan untuk menerima nilai dari luar fungsi ketika fungsi dipanggil.
4. Blok kode fungsi (isi fungsi)
   Isi fungsi harus ditulis dengan indentasi (menjorok ke dalam), biasanya satu tab atau empat spasi.
5. Perintah return
   Digunakan untuk mengembalikan nilai hasil proses dalam fungsi kepada bagian program yang memanggil fungsi tersebut.

Argument dan Parameter :
Dalam penggunaan fungsi terdapat dua istilah penting, yaitu argument dan parameter.

- Parameter adalah variabel yang terdapat pada definisi fungsi.
- Argument adalah nilai yang diberikan ketika fungsi dipanggil.

Sebagai contoh : tambah(5, 3)
Pada contoh tersebut:

- Angka 5 dan 3 merupakan argument.
- Variabel a dan b pada fungsi merupakan parameter yang akan menerima nilai argument tersebut.

MATERI 2 : Return Value

Return value merupakan nilai yang dihasilkan atau dikembalikan oleh suatu fungsi setelah fungsi tersebut dijalankan. Berdasarkan hasil yang dikeluarkan, fungsi dalam Python dapat dibedakan menjadi dua jenis, yaitu fungsi yang tidak mengembalikan nilai (void function) dan fungsi yang mengembalikan nilai.

1. Fungsi yang Tidak Mengembalikan Nilai (Void Function)
   Fungsi yang tidak mengembalikan nilai disebut **void function**. Fungsi ini hanya menjalankan perintah tertentu tanpa memberikan hasil yang dapat digunakan kembali oleh program.
   Contoh fungsi yang tidak mengembalikan nilai adalah sebagai berikut:

```
def print_twice(message):
    print(message)
    print(message)

print_twice("Hello World!")
```

   Pada fungsi tersebut terdapat satu parameter yaitu message. Ketika fungsi dipanggil, nilai yang diberikan pada parameter akan ditampilkan sebanyak dua kali menggunakan perintah print(). Jika fungsi tersebut dipanggil di dalam fungsi print(), maka akan terlihat bahwa fungsi tersebut sebenarnya mengembalikan nilai None.
   Contoh :

```
def print_twice(message):
    print(message)
    print(message)

print(print_twice("Hello World!"))
```

   Output yang dihasilkan:

```
Hello World!
Hello World!
None
```

   Nilai None menunjukkan bahwa fungsi tersebut tidak menghasilkan nilai tertentu yang dapat digunakan dalam proses perhitungan atau pengolahan data berikutnya.

2. Fungsi yang Mengembalikan Nilai
   Berbeda dengan void function, fungsi yang mengembalikan nilai menggunakan perintah return untuk mengirimkan hasil dari fungsi tersebut.
   Contoh fungsi yang mengembalikan nilai:

```
def tambah(a, b, c):
    hasil = a + b + c
    return hasil
```

   Pada fungsi tersebut terdapat tiga parameter yaitu a, b, dan c. Di dalam fungsi dilakukan proses penjumlahan terhadap ketiga nilai tersebut dan hasilnya disimpan

dalam variabel hasil. Selanjutnya variabel hasil dikembalikan sebagai nilai keluaran fungsi menggunakan perintah return.

3. Fungsi dari Keyword return
   Dalam Python, keyword return memiliki dua fungsi utama, yaitu:
   - Mengembalikan nilai dari fungsi
     Nilai yang dihasilkan oleh proses dalam fungsi dapat dikirim kembali ke program utama.
   - Mengakhiri proses eksekusi fungsi
     Setelah perintah return dijalankan, fungsi akan langsung berhenti dan tidak menjalankan kode setelahnya.

MATERI 3 : Optional Argument dan Named Argument

Dalam pemrograman Python, sebuah fungsi dapat memiliki parameter yang bersifat opsional, Parameter ini disebut **optional argument**. Selain itu, Python juga menyediakan cara pemanggilan fungsi dengan menyebutkan nama parameternya secara langsung. Cara ini disebut **named argument**.

1. Optional Argument
   Optional argument adalah parameter dalam fungsi yang memiliki **nilai default**. Parameter ini tidak wajib diisi ketika fungsi dipanggil karena sudah memiliki nilai bawaan yang akan digunakan jika tidak ada nilai yang diberikan.
   Contoh fungsi dengan optional argument:

```
def hitung_belanja(belanja, diskon=0):
  bayar = belanja - (belanja * diskon)/100
  return bayar
```

   Pada fungsi tersebut terdapat dua parameter, yaitu:
   - belanja → parameter utama yang harus diisi.
   - diskon → parameter opsional yang memiliki nilai default 0.

   Parameter diskon berarti jika pengguna tidak memasukkan nilai diskon, maka secara otomatis diskon dianggap 0%.

   Contoh pemanggilan fungsi:

```
print(hitung_belanja(100000))
print(hitung_belanja(100000, 10))
print(hitung_belanja(100000, 50))
```

Hasil output:

```
100000.0
90000.0
50000.0
```

Penjelasan:

- Pemanggilan pertama hanya memberikan nilai belanja, sehingga nilai diskon menggunakan default 0%.
- Pemanggilan kedua menggunakan diskon 10%.
- Pemanggilan ketiga menggunakan diskon 50%.

Dengan adanya optional argument, fungsi menjadi lebih fleksibel karena dapat digunakan dengan jumlah argument yang berbeda.

2. Named Argument

   Setiap parameter dalam fungsi memiliki nama. Oleh karena itu, ketika memanggil fungsi, kita juga dapat menyebutkan nama parameter tersebut secara langsung. Cara ini disebut **named argument**.

   Contoh fungsi:

```
def cetak(a, b, c):
    print("Nilai a: ", a)
    print("Nilai b: ", b)
    print("Nilai c: ", c)

cetak(20, 30, 40)
```

   Output:

```
Nilai a: 20
Nilai b: 30
Nilai c: 40
```

   nilai argument diberikan berdasarkan urutan parameter.

Keuntungan utama dari named argument adalah urutan argument tidak harus sama dengan urutan parameter pada fungsi.

MATERI 4 : Anonymous Function (Lambda)

Anonymous function merupakan fungsi yang tidak memiliki nama. Dalam Python, anonymous function dibuat menggunakan keyword **lambda**. Fungsi ini biasanya digunakan untuk operasi sederhana yang tidak memerlukan definisi fungsi secara lengkap menggunakan def.

Penggunaan anonymous function dalam Python hanya sebagai fitur tambahan. Fungsi lambda biasanya digunakan untuk membuat fungsi sederhana secara singkat tanpa perlu membuat fungsi dengan nama tertentu.

Struktur dasar anonymous function terdiri dari keyword lambda, parameter (argument), dan ekspresi yang menghasilkan suatu nilai. Fungsi lambda memungkinkan penulisan fungsi secara lebih ringkas dibandingkan dengan fungsi biasa yang menggunakan def.

Sebagai contoh, berikut adalah fungsi biasa yang digunakan untuk menjumlahkan dua bilangan:

```
def tambah(a, b):

   hasil = a + b

   return hasil

print(tambah(10,20))
```

Penjelasan:

- Fungsi tambah() memiliki dua parameter yaitu a dan b.
- Kedua nilai tersebut dijumlahkan dan hasilnya disimpan dalam variabel hasil.
- Nilai tersebut kemudian dikembalikan menggunakan perintah return.

Menggunakan Lambda Function

Fungsi yang sama dapat dituliskan menggunakan lambda function dengan cara yang lebih singkat:

```
tambah = lambda a, b: a + b
print(tambah(10,20))
```

Penjelasan:

- Fungsi tidak menggunakan def.
- Fungsi langsung dituliskan dalam satu baris.
- Nilai hasil operasi langsung menjadi nilai yang dikembalikan oleh fungsi.

Dengan cara ini, fungsi menjadi lebih sederhana dan ringkas.

Secara umum, lambda function pada Python terdiri dari beberapa bagian utama, yaitu:

1. Keyword lambda
   Digunakan untuk mendefinisikan anonymous function.
2. Bound Variable
   Merupakan parameter atau argument yang digunakan dalam fungsi lambda.
3. Body (Ekspresi)
   Bagian utama fungsi yang berisi operasi atau ekspresi yang akan menghasilkan suatu nilai.

Beberapa kelebihan penggunaan anonymous function, Penulisan fungsi menjadi lebih singkat, Cocok untuk operasi sederhana, Tidak perlu membuat fungsi dengan nama khusus. Namun lambda function juga memiliki keterbatasan, yaitu hanya dapat berisi satu ekspresi dan tidak cocok untuk logika program yang kompleks.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

Buatlah sebuah fungsi yang dapat menentukan apakah ketiga parameter memenuhi semua ketentuan berikut ini:

• Ketiga parameter tersebut nilainya berbeda semua.

• Ada kemungkinan jika diambil dua parameter dan dijumlahkan hasilnya sama dengan parameter lainnya (yang tersisa).

Fungsi tersebut akan menghasilkan nilai True jika semua ketentuan tersebut dipenuhi. Jika tidak terpenuhi maka fungsi akan menghasilkan nilai False. Fungsi anda harus diberi nama cek_angka().

```python
soal01.py
1  def cek_angka(a, b, c):
2      if a != b and a != c and b != c:
3          if (a + b == c) or (a + c == b) or (b + c == a):
4              return True
5      return False
6
7  # Tes
8  print(cek_angka(2, 3, 5))
9  print(cek_angka(4, 4, 8))
10 print(cek_angka(1, 2, 4))
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS E:\Prak_Alpro\minggu05> & "C:\Users\ALFA DELTA\AppData
o/minggu05/soal01.py
True
False
False
PS E:\Prak_Alpro\minggu05>
```

- Fungsi dibuat menggunakan def dengan nama cek_angka() yang memiliki tiga parameter yaitu a, b, dan c.
- Pertama memeriksa apakah ketiga nilai tersebut berbeda dengan kondisi: a != b and a != c and b != c
- Program kemudian memeriksa apakah terdapat dua angka yang jika dijumlahkan menghasilkan angka yang ketiga. Baris ke 3.
- Jika kedua syarat terpenuhi, fungsi mengembalikan nilai True.
- Jika salah satu syarat tidak terpenuhi, maka fungsi mengembalikan False.
- cek_angka(2, 3, 5) → **True** karena semua angka berbeda dan 2 + 3 = 5.
- cek_angka(4, 4, 8) → **False** karena ada dua angka yang sama.
- cek_angka(1, 2, 4) → **False** karena tidak ada dua angka yang jumlahnya sama dengan angka lainnya.

SOAL 2

Buatlah sebuah fungsi yang dapat menentukan apakah minimal dua dari tiga parameter yang diberikan memiliki digit paling kanan yang sama. Fungsi tersebut menghasilkan nilai True jika memenuhi dan False jika tidak memenuhi. Gunakan fungsi tersebut untuk mengecek beberapa test-case berikut ini:

- Input = 30, 20, 18. Output yang diharapkan = True
- Input = 145, 5, 100. Output yang diharapkan = True
- Input = 71, 187, 18. Output yang diharapkan = False
- Input = 1024, 14, 94. Output yang diharapkan = True
- Input = 53, 8900, 658. Output yang diharapkan = False

Ketiga bilangan tersebut diinputkan oleh pengguna, sehingga anda perlu membaca input dari pengguna. Fungsi anda harus diberi nama cek_digit_belakang()

soal02.py

```python
def cek_digit_belakang(a, b, c):
    a1 = a % 10
    b1 = b % 10
    c1 = c % 10

    if (a1 == b1) or (a1 == c1) or (b1 == c1):
        return True
    else:
        return False

a = int(input("Masukkan bilangan pertama: "))
b = int(input("Masukkan bilangan kedua: "))
c = int(input("Masukkan bilangan ketiga: "))

print("Hasil:", cek_digit_belakang(a, b, c))
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS E:\Prak_Alpro\minggu05> & "C:\Users\ALFA DELTA\AppData\
o/minggu05/soal02.py
Masukkan bilangan pertama: 30
Masukkan bilangan kedua: 20
Masukkan bilangan ketiga: 18
Hasil: True
PS E:\Prak_Alpro\minggu05> & "C:\Users\ALFA DELTA\AppData\
o/minggu05/soal02.py
Masukkan bilangan pertama: 145
Masukkan bilangan kedua: 5
Masukkan bilangan ketiga: 100
Hasil: True
PS E:\Prak_Alpro\minggu05> & "C:\Users\ALFA DELTA\AppData\
o/minggu05/soal02.py
Masukkan bilangan pertama: 71
Masukkan bilangan kedua: 187
Masukkan bilangan ketiga: 18
Hasil: False
PS E:\Prak_Alpro\minggu05> & "C:\Users\ALFA DELTA\AppData\
o/minggu05/soal02.py
Masukkan bilangan pertama: 1024
Masukkan bilangan kedua: 14
Masukkan bilangan ketiga: 94
Hasil: True
PS E:\Prak_Alpro\minggu05> & "C:\Users\ALFA DELTA\AppData\
o/minggu05/soal02.py
Masukkan bilangan pertama: 53
Masukkan bilangan kedua: 8900
Masukkan bilangan ketiga: 658
Hasil: False
PS E:\Prak_Alpro\minggu05>
```

- Fungsi dibuat dengan nama cek_digit_belakang(a, b, c) yang menerima tiga parameter.
- Digit terakhir dari setiap bilangan diperoleh menggunakan operator % 10.
- Pengecekan kondisi, apakah tiap digit sama atau tidak. Baris ke 6
- Jika salah satu kondisi terpenuhi → True
- Jika tidak → False

SOAL 3

Buatlah fungsi-fungsi konversi suhu menggunakan lambda function. Fungsi-fungsi yang harus anda implementasikan:

• Celcius to Fahrenheit. F = (9/5) ∗C +32
• Celcius to Reamur. R = 0.8 ∗C

Berikan contoh penggunaannya untuk test-case berikut ini:

• Input C = 100. Output F = 212.
• Input C = 80. Output R = 64.
• Input = 0. Output F = 32.

soal03.py

```python
celcius_to_fahrenheit = lambda C: int((9/5) * C + 32)
celcius_to_reamur = lambda C: int(0.8 * C)

# Tes
print("Input C = 100, Output F =", celcius_to_fahrenheit(100))
print("Input C = 80, Output R =", celcius_to_reamur(80))
print("Input C = 0, Output F =", celcius_to_fahrenheit(0))
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS E:\Prak_Alpro\minggu05> & "C:\Users\ALFA DELTA\AppDat
o/minggu05/soal03.py
Input C = 100, Output F = 212
Input C = 80, Output R = 64
Input C = 0, Output F = 32
PS E:\Prak_Alpro\minggu05>
```

- Membuat lambda function, celcius_to_fahrenheit digunakan untuk mengubah suhu dari Celcius ke Fahrenheit. celcius_to_reamur digunakan untuk mengubah suhu dari Celcius ke Reamur.
- Parameter C merupakan nilai suhu dalam Celcius yang akan dikonversi.
- Perhitungan dalam lambda:
- (9/5) * C + 32 digunakan untuk menghitung Fahrenheit.
- 0.8 * C digunakan untuk menghitung Reamur.

Github : https://github.com/71200556/71200556_frans